STUDENT PROJECT TEMPLATE (SRS)

(INSTRUCTION FOR STUDENTS: THIS DOCUMENT SERVES AS THE FORMAL 'CONTRACT' FOR YOUR TEAM PROJECT. IT MUST BE MAINTAINED IN YOUR GITHUB REPOSITORY.)

OPENANTHROPIC (FINANCE BUDGET TRACKER)

SOFTWARE REQUIREMENTS SPECIFICATION

INSTRUCTOR: LARA NICHOLS-BROWN

TERM: FALL 2026

SECTION: 11

THE ENGINEERING TEAM:

1. **STUDENT A:** SHREEV B. — LEAD DEVELOPER
2. **STUDENT B:** JUAN S. — SCRUM MASTER & PROJECT MANAGER
3. **STUDENT C:** JUAN F. — PROJECT TESTER

PROJECT ASSETS:

- **GITHUB:** https://github.com/shreev2703/finance-budget-tracker
- **DEPLOYMENT:** TBD

DOCUMENT HISTORY:

LAST DATE CHANGED:	WHO:	WHAT WAS CHANGED:
09/18/2026	SHREEV B., JUAN S., JUAN F.	COMPLETE TE1 MONOREPO SETUP
09/25/2026	SHREEV B., JUAN S., JUAN F.	ADD TE2 SRS DOCUMENT WITH PRODUCT VISION AND USER STORIES
10/02/2026	SHREEV B.	TE3: ADD UML CLASS DIAGRAM, DATABASE SCHEMA, AND REST

		API ENDPOINTS
10/02/2026	JUAN S.	TE3: ADD UI MOCKUPS AND INITIAL UML SKETCH
10/07/2026	JUAN S.	SET UP REACT FRONTEND PROTOTYPE
10/16/2026	JUAN F.	BUILD WORKING WEBSITE PROTOTYPE (LOGIN, DASHBOARD, TRANSACTIONS, CATEGORIES)
10/16/2026	SHREEV B.	TE4: ADD ARCHITECTURE, DESIGN PATTERN, AND DESIGN DECISIONS; COMPLETE FUNCTIONAL REQUIREMENTS, NON-FUNCTIONAL REQUIREMENTS, DATA REQUIREMENTS, AND TRACEABILITY MATRIX
10/23/2026	SHREEV B., JUAN S., JUAN F.	TE5: CODE REVIEW AND REFACTORING TICKETS R-01 TO R-06 (#50–#55); R-01, R-04, AND R-05 SELECTED FOR THE REFACTORING SPRINT
10/23/2026	SHREEV B.	SPRINT 3: CONNECT THE FRONTEND TO THE API (VITE PROXY, API CLIENT, ONE-COMMAND START); ADD BUDGETS API (POST/PUT /api/budgets) FOR US-08
10/23/2026	JUAN F.	SPRINT 3: DASHBOARD SHOWS LIVE TOTALS, BUDGET BARS WITH THE 80% WARNING, AND TRANSACTIONS FROM THE API (#43, #44)

10/23/2026	SHREEV B.	TE5 R-01: MOVE TRANSACTION VALIDATION INTO domain/transaction.js; UPDATE TRACEABILITY MATRIX LINE REFERENCES
10/23/2026	SHREEV B.	SPRINT 3: ADD TRANSACTION FORM THAT SAVES TO POST /api/transactions (US-01, US-02); CREATE / EDIT CATEGORY NOW SAVE THROUGH THE API (US-08); ADD POST, PUT, AND DELETE /api/categories
10/23/2026	SHREEV B.	TE5 R-05: SHARE ONE CATEGORY FORM AND ONE TRANSACTION ROW IN THE FRONTEND
10/23/2026	SHREEV B.	TE5 R-04: SPLIT App.jsx INTO PAGE AND COMPONENT FILES; UPDATE TRACEABILITY MATRIX FOR THE NEW FILES
10/27/2026	SHREEV B.	WEEKLY UPDATE: R-04 AND R-05 MERGED INTO MAIN (#61); RE-CHECKED TRACEABILITY MATRIX LINE REFERENCES AGAINST MAIN
10/28/2026	JUAN F.	SPRINT 4: FRONTEND DEMO DATA STORED IN THE BROWSER, WITH THE OPTION TO START WITH OR WITHOUT SAMPLE DATA (#63, #64); FIX A BLANK-SCREEN BUG IN THE VITE CONFIG. SWITCHING BACK TO THE LIVE API BY DEFAULT IS #67

10/29/2026	SHREEV B.	SPRINT 4: EDIT AND DELETE A TRANSACTION (US-03, FR-03) WITH PUT AND DELETE /api/transactions/:id AND EDIT / DELETE BUTTONS ON EACH TRANSACTION ROW (#68); UPDATE TRACEABILITY MATRIX
11/02/2026	SHREEV B.	WEEKLY UPDATE FOR OCT 26–30: US-03 MERGED INTO MAIN (#72); RE-CHECKED TRACEABILITY MATRIX LINE REFERENCES AGAINST MAIN
11/06/2026	SHREEV B.	TE6: ACCESS CONTROL (US-07, FR-07): SIGN-UP AND SIGN-IN ENDPOINTS, BCRYPT PASSWORD HASHING, JWT TOKENS, AND AN AUTH CHECK ON EVERY DATA ROUTE; LOGIN AND SIGN-UP SCREENS; TOKEN STORED IN THE BROWSER AND SENT WITH EVERY API CALL; SEQUENCE DIAGRAMS IN SECTION 5.5 (#77, #78, #79)
11/06/2026	SHREEV B.	THE FRONTEND USES THE LIVE API BY DEFAULT AGAIN; JUAN F.'S BROWSER DEMO DATA IS KEPT BEHIND VITE_DEMO=1 (#67)
11/06/2026	SHREEV B.	UNPLANNED FIXES NEEDED TO DEMO THE APP (#80): DEMO ACCOUNT WAS EMPTY AFTER THE MONTH CHANGED; NEW ACCOUNTS COULD NOT SET A LIMIT ON THEIR STARTING

		CATEGORIES; NEGATIVE BALANCES SHOWED AS $-85.00; HARD-CODED 2025 DATES; REMOVED LOGIN BUTTONS THAT DID NOTHING
11/06/2026	SHREEV B.	TE1 AND TE4 FOLLOW-UPS: WROTE CONTRIBUTING.MD (#81); ADDED THE FRONTEND ARCHITECTURE DOCUMENT (#26) AND THE ADD-TRANSACTION SEQUENCE DIAGRAM (#27)
11/06/2026	SHREEV B.	WEEKLY UPDATE FOR NOV 2–6 (#75): RE-CHECKED ALL TRACEABILITY MATRIX LINE REFERENCES; NEW LOGIN AND DASHBOARD SCREENSHOTS
11/06/2026	SHREEV B.	SPRINT 5: FILTER TRANSACTION HISTORY BY CATEGORY AND DATE RANGE (US-06, FR-06, #74); SPENDING-BY-CATEGORY CHART ON THE DASHBOARD (US-05, FR-05, #76). EVERY USER STORY NOW HAS WORKING CODE
11/06/2026	SHREEV B.	REFACTORING BACKLOG: MOVE currentPeriod() TO domain/period.js (R-02, #51); SHARED TRANSACTION SELECT AND NAMED PAGING LIMITS (R-03, #52); PAGE LOOKUP MAP AND PAGES CONSTANT (R-06, #55). TAKEN OVER FROM JUAN S.: R-06, CI, AND DEMO MATERIALS

11/06/2026	JUAN S., SHREEV B.	ADD OVERVIEW SEQUENCE DIAGRAM: FIRST DRAFT BY JUAN S.; REDRAWN BY SHREEV B. FROM THE REQUESTS THE RUNNING APP MAKES, WITH THE BUILT EMAIL-AND-PASSWORD LOGIN AND API ROUTES
11/06/2026	SHREEV B.	UNPLANNED FIXES FROM END-TO-END TESTING (#80): API RESPONSES NO LONGER CACHED IN THE BROWSER; BUDGET CARDS NO LONGER OVERFLOW ON PHONES; "MONTHLY BUDGET" RELABELED "MONTHLY INCOME"; REMOVED THE TUTORIAL HELP BUTTON THAT DID NOTHING; APP BRANDING CHANGED FROM CAL POLY TO LEDGERLOGIC; FIXED ALL LINT WARNINGS
11/06/2026	SHREEV B.	ADD CI: LINT, TESTS, AND THE FRONTEND BUILD RUN ON EVERY PULL REQUEST (#28); ADD DEMO.MD AND BACKUP SCREENSHOTS (#69); UPDATE TRACEABILITY MATRIX AND SCREENSHOTS

LEDGERLOGIC (FINANCE BUDGET TRACKER)

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

COURSE: CSC 3100 — INTRODUCTION TO SOFTWARE ENGINEERING

DATE: NOVEMBER 6, 2026

TABLE OF CONTENTS

1. **INTRODUCTION ..... [PAGE #]**
    - **1.1 PROJECT PURPOSE**
    - **1.2 INTENDED AUDIENCE**
    - **1.3 PROJECT SCOPE**
2. **USER STORIES ..... [PAGE #]**
    - **2.1 USER FEATURES (THE "SHALL" STATEMENTS)**
    - **2.2 ADMIN FEATURES**
3. **FUNCTIONAL REQUIREMENTS..... [PAGE #]**
4. **NON-FUNCTIONAL REQUIREMENTS ..... [PAGE #]**
    - **4.1 DATA INTEGRITY & SECURITY**
    - **4.2 PERFORMANCE & USABILITY**
5. **SYSTEM ARCHITECTURE ..... [PAGE #]**
    - **5.1 REST API ENDPOINTS**
    - **5.2 DATABASE SCHEMA (MYSQL)**
    - **5.3 UML CLASS DIAGRAM**
    - **5.4 ARCHITECTURE & DESIGN PATTERNS**
    - **5.5 ACCESS CONTROL**
6. **USER INTERFACE (UI) ..... [PAGE #]**
    - **6.1 WIREFRAMES / MOCKUPS**
7. **DATA REQUIREMENTS ..... [PAGE #]**
    - **7.1 LIST OF PERSISTENT DATA**
8. **TRACEABILITY MATRIX ..... [PAGE #]**
9. **APPENDICES ..... [PAGE #]**
10. **AI USAGE & DISCLOSURE (MANDATORY) ..... [PAGE #]**

---

1. INTRODUCTION

1.1 PROJECT PURPOSE

- **PROBLEM STATEMENT:** COLLEGE STUDENTS AND YOUNG ADULTS OFTEN LOSE TRACK OF SMALL DAILY PURCHASES AND OVERSPEND THEIR MONTHLY ALLOWANCE OR PART-TIME INCOME. EXISTING OPTIONS ARE EITHER TEDIOUS MANUAL SPREADSHEETS OR PAID APPS THAT REQUIRE LINKING A BANK ACCOUNT.
- **PRODUCT VISION:** FOR COLLEGE STUDENTS AND YOUNG ADULTS WHO STRUGGLE TO TRACK DAILY EXPENSES AND STICK TO A MONTHLY ALLOWANCE, LEDGERLOGIC IS A PERSONAL FINANCE AND BUDGET TRACKING WEB APPLICATION WHICH SIMPLIFIES DAILY TRANSACTION LOGGING AND PROVIDES INSTANT VISUAL INSIGHTS INTO SPENDING HABITS. UNLIKE COMPLEX MANUAL SPREADSHEETS OR PAID SUBSCRIPTION APPS THAT REQUIRE INVASIVE BANK ACCOUNT LINKING, OUR PRODUCT OFFERS A FAST, CLUTTER-FREE INTERFACE FOR MANUAL EXPENSE TRACKING AND GOAL SETTING WITHOUT SUBSCRIPTION FEES OR PRIVACY RISKS.

1.2 INTENDED AUDIENCE

- **TARGET AUDIENCE:** PRIMARY USERS ARE COLLEGE STUDENTS AND YOUNG ADULTS MANAGING A FIXED MONTHLY ALLOWANCE OR PART-TIME INCOME. STAKEHOLDERS INCLUDE THESE END USERS, THE LEDGERLOGIC ENGINEERING TEAM, AND THE CSC 3100 INSTRUCTOR AS PROJECT CLIENT AND EVALUATOR.

1.3 PROJECT SCOPE

- **SCOPE:** IN SCOPE: ACCOUNT REGISTRATION AND LOGIN; MANUAL LOGGING, EDITING, AND DELETING OF INCOME AND EXPENSE TRANSACTIONS; A DASHBOARD WITH TOTALS AND REMAINING BALANCE; A SPENDING-BY-CATEGORY CHART; FILTERED TRANSACTION HISTORY; AND PER-CATEGORY MONTHLY SPENDING LIMITS WITH AN 80% WARNING. OUT OF SCOPE: BANK ACCOUNT LINKING, AUTOMATIC TRANSACTION IMPORT, MULTI-CURRENCY SUPPORT, SHARED OR HOUSEHOLD BUDGETS, AND NATIVE MOBILE APPS.

2. USER STORIES

USER STORIES FOLLOW THE FORMAT: ***"AS A [TYPE OF USER], I WANT TO [ACTION] SO THAT [VALUE/BENEFIT]."****...*

2.1 USER FEATURES (THE "SHALL" STATEMENTS)

- **US-01:** LOG AN EXPENSE — MAIN AUTHOR: SHREEV B.
- **US-02:** RECORD INCOME — MAIN AUTHOR: SHREEV B.
- **US-03:** EDIT / DELETE TRANSACTIONS — MAIN AUTHOR: SHREEV B.
- **US-04:** DASHBOARD SUMMARY — MAIN AUTHOR: JUAN F.
- **US-05:** SPENDING-BY-CATEGORY CHART — MAIN AUTHOR: JUAN F.
- **US-06:** FILTER TRANSACTION HISTORY — MAIN AUTHOR: JUAN F.
- **US-07:** ACCOUNT & AUTHENTICATION — MAIN AUTHOR: JUAN S.
- **US-08:** CATEGORY SPENDING LIMITS — MAIN AUTHOR: JUAN S.
- **US-09:** BUDGET WARNING AT 80% — MAIN AUTHOR: JUAN S.

ID	MAIN AUTHOR	REQUIREMENT	PRIORITY
US-01	SHREEV B.	As a budget-conscious student, I want to log a new expense with an amount, date, category, and description so that I can track where my money goes each day.	1
US-02	SHREEV B.	As a working student, I want to record my paychecks and income sources so that my total available monthly funds are accurately calculated.	1
US-03	SHREEV B.	As a registered user, I want to edit or delete a past transaction so that I can fix typos or remove duplicate entries.	2
US-04	JUAN F.	As a daily user, I want to view a home dashboard showing my total income, total expenses, and remaining balance so that I can see my financial standing at a glance.	1
US-05	JUAN F.	As a visual planner, I want to see a chart breaking down my spending by category so that I can identify	2

		which areas consume most of my budget.	
US-06	JUAN F.	As a returning user, I want to filter my transaction history by category and date range so that I can quickly find specific past purchases.	3
US-07	JUAN S.	As a privacy-conscious user, I want to create an account and log in with an email and password so that my financial records stay private and saved across sessions.	1
US-08	JUAN S.	As a user on a strict budget, I want to set a monthly spending limit for individual categories so that I can keep my discretionary spending under control.	2
US-09	JUAN S.	As a proactive saver, I want to see a visual warning when my spending exceeds 80% of a category's limit so that I can slow down before going over budget.	3

2.2 ADMIN FEATURES

- NONE IN THE CURRENT SCOPE. LEDGERLOGIC IS A SINGLE-USER PERSONAL TOOL; EVERY USER MANAGES ONLY THEIR OWN DATA.

3. FUNCTIONAL REQUIREMENTS

THE SYSTEM SHALL...

ID	REQUIREMENT	PRIORITY
FR-01	Allow a user to log an expense with an amount, date, category, and description. (US-01)	1
FR-02	Allow a user to record income with an amount, date, income source, and description. (US-02)	1
FR-03	Allow a user to edit or delete any of their own past transactions. (US-03)	2
FR-04	Show a dashboard with the current month's total income, total expenses, and remaining balance. (US-04)	1

FR-05	Show a chart of the current month's spending broken down by category. (US-05)	2
FR-06	Filter the transaction history by category and by date range. (US-06)	3
FR-07	Authenticate users with an email and password, and show each user only their own data. (US-07)	1
FR-08	Let a user set a monthly spending limit for each expense category and show the amount spent and remaining. (US-08)	2
FR-09	Show a visual warning when spending in a category reaches 80% of its limit, and an over-limit state above 100%. (US-09)	3

4. NON-FUNCTIONAL REQUIREMENTS

4.1 DATA INTEGRITY & SECURITY

- **INTEGRITY:** HANDLING ERRORS AND BAD DATA. THE API REJECTS INVALID INPUT WITH A CLEAR LIST OF ERRORS (HTTP 400): AMOUNTS MUST BE POSITIVE AND AT MOST $1,000,000, DATES MUST BE VALID (YYYY-MM-DD), AND A TRANSACTION'S CATEGORY MUST BELONG TO THE USER AND MATCH THE TRANSACTION TYPE. THE DATABASE ENFORCES THE SAME RULES WITH CHECK AND FOREIGN KEY CONSTRAINTS. MONEY IS STORED AS INTEGER CENTS TO AVOID ROUNDING ERRORS.
- **SECURITY:** PASSWORDS ARE HASHED WITH BCRYPT AND NEVER RETURNED BY THE API. EVERY DATA ROUTE REQUIRES A SIGNED JWT THAT EXPIRES AFTER ONE DAY, AND THE SIGNING KEY COMES FROM THE TOKEN_SECRET ENVIRONMENT VARIABLE, NOT THE CODE. EVERY QUERY IS SCOPED TO THE LOGGED-IN USER, AND ALL SQL USES PARAMETERIZED QUERIES TO PREVENT SQL INJECTION. API RESPONSES ARE SENT WITH CACHE-CONTROL: NO-STORE SO FINANCIAL DATA DOES NOT STAY IN THE BROWSER CACHE.

4.2 PERFORMANCE & USABILITY

- **PERFORMANCE:** THE DASHBOARD LOADS ITS TOTALS AND BUDGET STATUS FROM A SINGLE API CALL, AND API RESPONSES SHOULD RETURN IN UNDER 500 MS ON A LOCAL MACHINE.

- **USABILITY:** LOGGING A TRANSACTION TAKES ONE SHORT FORM (AMOUNT, CATEGORY, DATE, OPTIONAL NOTE). ERRORS ARE SHOWN IN PLAIN LANGUAGE, BUDGET STATUS USES BOTH COLOR AND TEXT (E.G. "OVER BUDGET"), AND THE LAYOUT WORKS ON DESKTOP AND PHONE-WIDTH SCREENS.

5. SYSTEM ARCHITECTURE

5.1 REST API ENDPOINTS

- **REST ENDPOINTS:** (LIST METHOD | URL | DESCRIPTION) — BASE URL: http://localhost:3001/api
- **PUBLIC ENDPOINTS:**
  - **POST /api/auth/register — CREATES AN ACCOUNT AND RETURNS A TOKEN (400 IF INVALID, 409 IF THE EMAIL IS TAKEN).**
  - **POST /api/auth/login — CHECKS THE HASHED PASSWORD AND RETURNS A TOKEN (401 IF WRONG).**
  - **GET /api/health — CONFIRMS THE API IS RUNNING.**
- **PROTECTED ENDPOINTS (REQUIRE AUTHORIZATION: BEARER TOKEN; 401 WITHOUT A VALID ONE):**
  - **GET /api/auth/me — FETCHES THE LOGGED-IN USER.**
  - **GET /api/transactions?categoryId=&from=&to= — FETCHES THE USER'S TRANSACTIONS, NEWEST FIRST, WITH OPTIONAL FILTERS (400 FOR A BAD DATE OR AN UNKNOWN CATEGORY).**
  - **POST /api/transactions — CREATES A NEW INCOME OR EXPENSE (RETURNS 400 WITH A LIST OF ERRORS IF INVALID).**
  - **PUT /api/transactions/:id — EDITS A TRANSACTION WITH THE SAME RULES AS POST (404 IF IT IS NOT THE USER'S).**
  - **DELETE /api/transactions/:id — DELETES A TRANSACTION (404 IF IT IS NOT THE USER'S).**
  - **GET /api/budgets/summary — FETCHES MONTHLY TOTALS AND EACH CATEGORY'S LIMIT, SPENT, REMAINING, AND STATUS (OK / WARNING / OVER).**
  - **GET /api/reports/spending-by-category — FETCHES THE MONTH'S SPENDING PER CATEGORY, LARGEST FIRST, FOR THE CHART.**
  - **GET /api/categories — FETCHES THE USER'S INCOME AND EXPENSE CATEGORIES WITH THIS MONTH'S LIMIT.**

- **POST /api/budgets — SETS A MONTHLY LIMIT FOR AN EXPENSE CATEGORY (409 IF ONE ALREADY EXISTS).**
- **PUT /api/budgets/:id — CHANGES A BUDGET'S MONTHLY LIMIT.**
- **POST /api/categories — CREATES AN INCOME OR EXPENSE CATEGORY (409 IF THE NAME IS ALREADY USED).**
- **PUT /api/categories/:id — RENAMES A CATEGORY.**
- **DELETE /api/categories/:id — DELETES A CATEGORY AND ITS BUDGETS (409 IF IT STILL HAS TRANSACTIONS).**

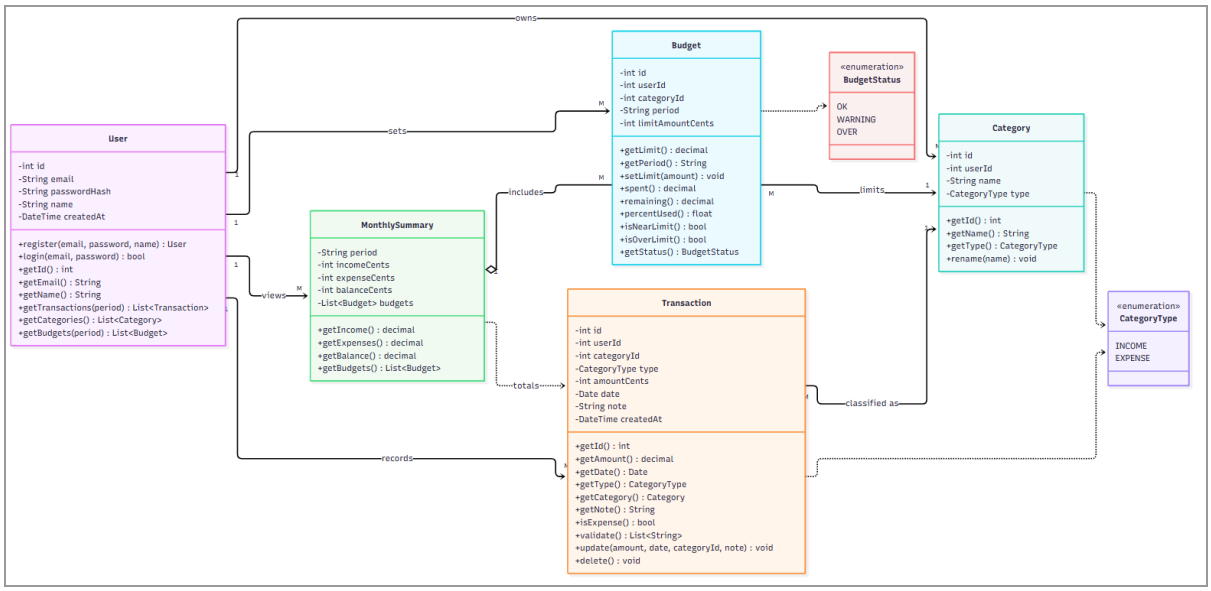
5.2 DATABASE SCHEMA (MYSQL)

- DEFINED IN packages/backend/src/db/schema.sql. DEVELOPMENT AND DEMOS USE SQLITE (NO DATABASE SERVER NEEDED); EVERY TABLE AND COLUMN MAPS ONE-TO-ONE TO MYSQL FOR PRODUCTION.

TABLE	COLUMNS	KEYS / CONSTRAINTS
users	id, email, password_hash, name, created_at	PK id; email UNIQUE
categories	id, user_id, name, type	PK id; FK user_id → users; type IN (INCOME, EXPENSE)
transactions	id, user_id, category_id, type, amount_cents, date, note, created_at	PK id; FK user_id → users; FK category_id → categories; amount_cents > 0
budgets	id, user_id, category_id, period (YYYY-MM), limit_amount_cents	PK id; FK user_id → users; FK category_id → categories; one per category per month

5.3 UML CLASS DIAGRAM

- **UML:** (ATTACH A DIAGRAM) — LEDGERLOGIC DOMAIN MODEL: USER, CATEGORY, TRANSACTION, BUDGET, AND THE CATEGORYTYPE ENUMERATION.



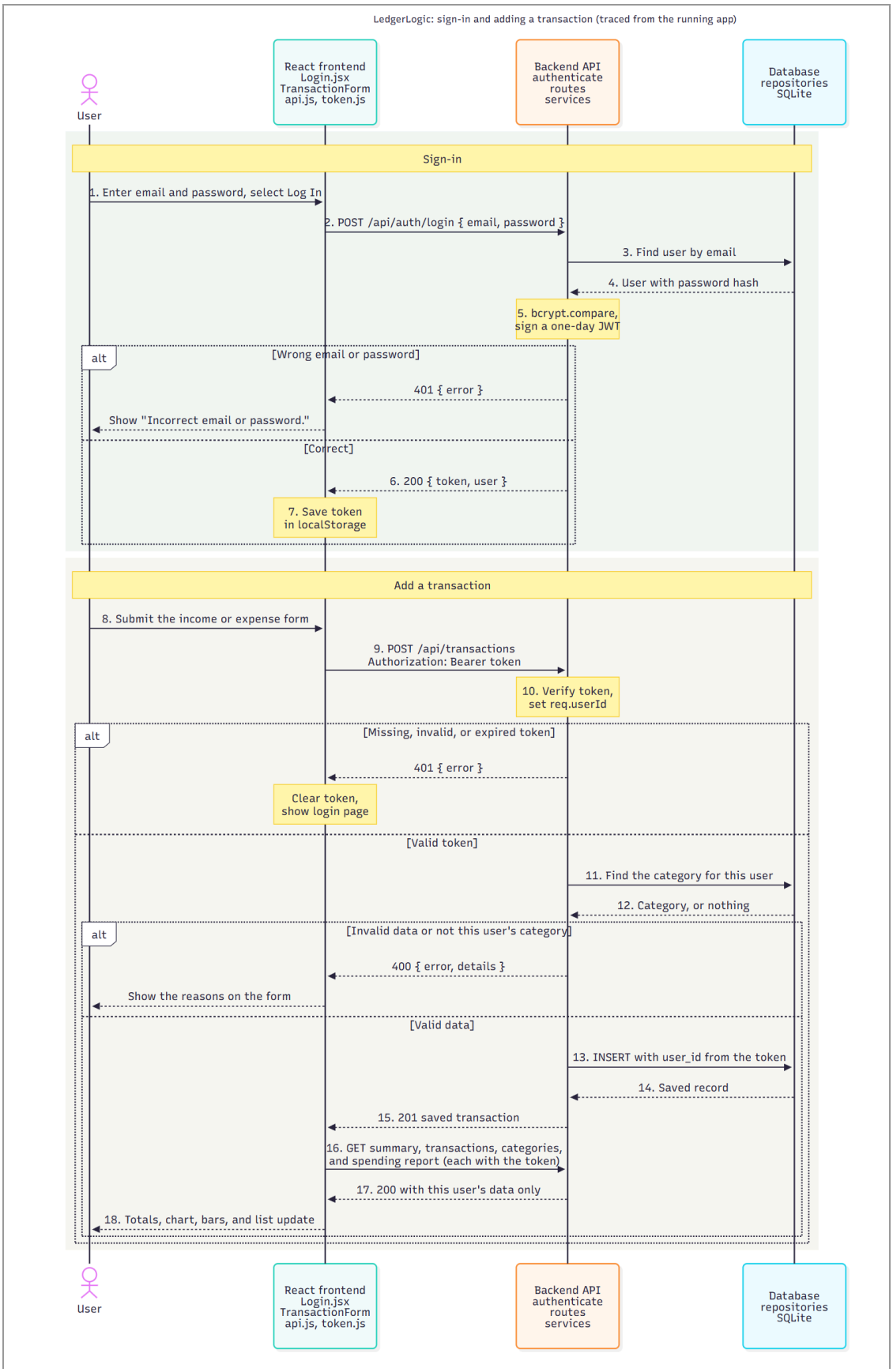
5.4 ARCHITECTURE & DESIGN PATTERNS

- **ARCHITECTURE:** LAYERED ARCHITECTURE INSIDE A CLIENT-SERVER SYSTEM: REACT CLIENT → EXPRESS MIDDLEWARE → ROUTES → SERVICES → REPOSITORIES → DATABASE. NO EXTERNAL SYSTEMS.
- **DESIGN PATTERN:** CHAIN OF RESPONSIBILITY (GANG OF FOUR), IMPLEMENTED AS THE EXPRESS MIDDLEWARE CHAIN.
- **DESIGN DECISIONS:** SERVICE AND REPOSITORY LAYERS; SQLITE NOW WITH A MYSQL-COMPATIBLE SCHEMA; MONEY STORED AS INTEGER CENTS; BCRYPT LOGIN BUILT AFTER THE CORE BUDGETING FLOW; SHARED REACT STATE IN THE TOP-LEVEL COMPONENT.
- **FRONTEND ARCHITECTURE (COMPONENT TREE AND STATE):** docs/architecture/frontend.md
- **FULL TE4 DOCUMENT (ALL TE4 DIAGRAMS):** docs/te4/TE4-Design-and-Architectural-Patterns.pdf

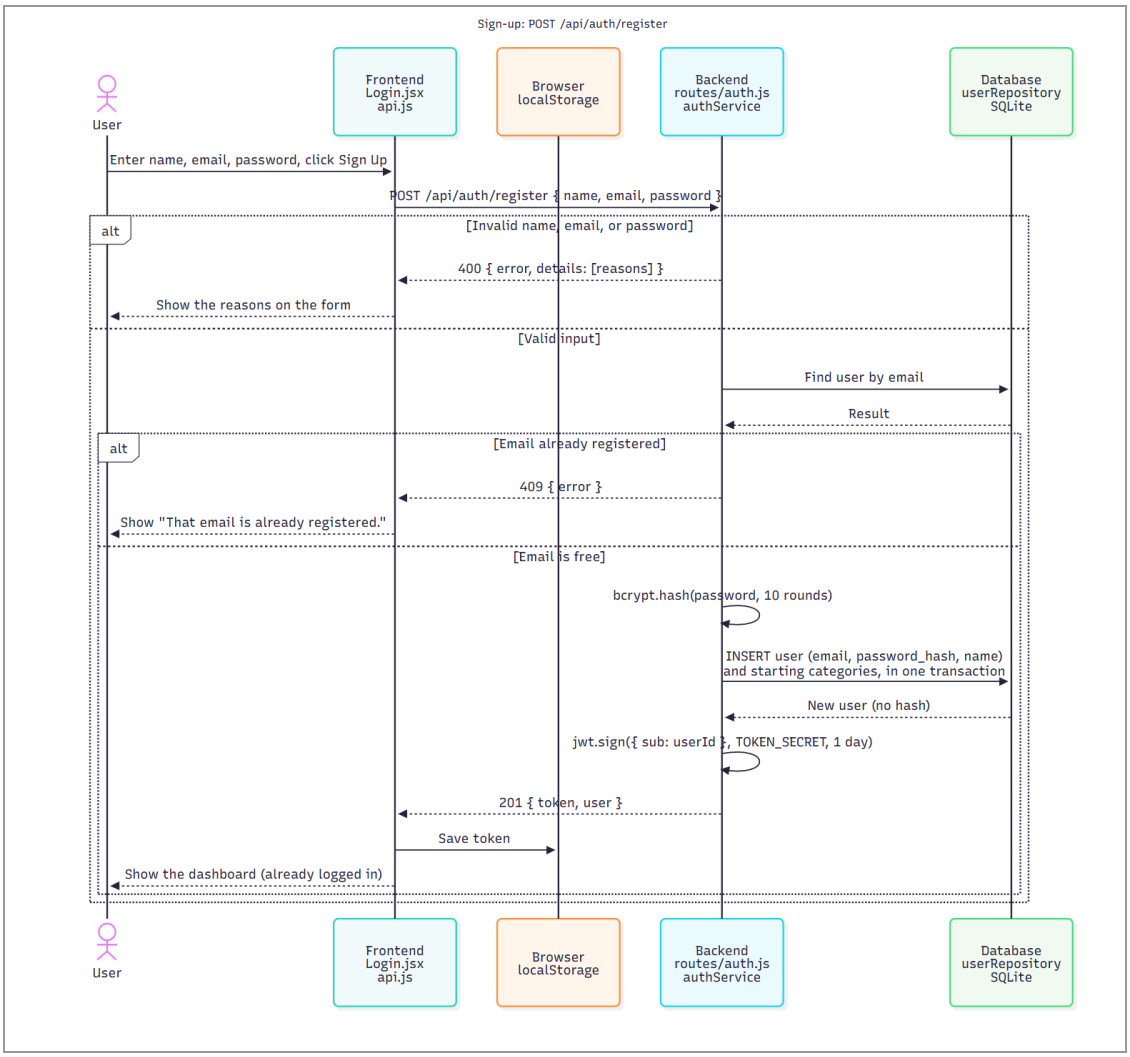
5.5 ACCESS CONTROL

- **DESIGN:** SIGN-UP AND SIGN-IN RETURN A JWT THAT EXPIRES AFTER ONE DAY. THE FRONTEND STORES IT IN LOCALSTORAGE SO A PAGE REFRESH STAYS LOGGED IN, AND api.js SENDS IT AS AN AUTHORIZATION: BEARER HEADER WITH EVERY CALL. THE AUTHENTICATE MIDDLEWARE VERIFIES IT BEFORE ANY DATA ROUTE RUNS AND TAKES THE USER ID FROM THE TOKEN, NEVER FROM THE REQUEST BODY.

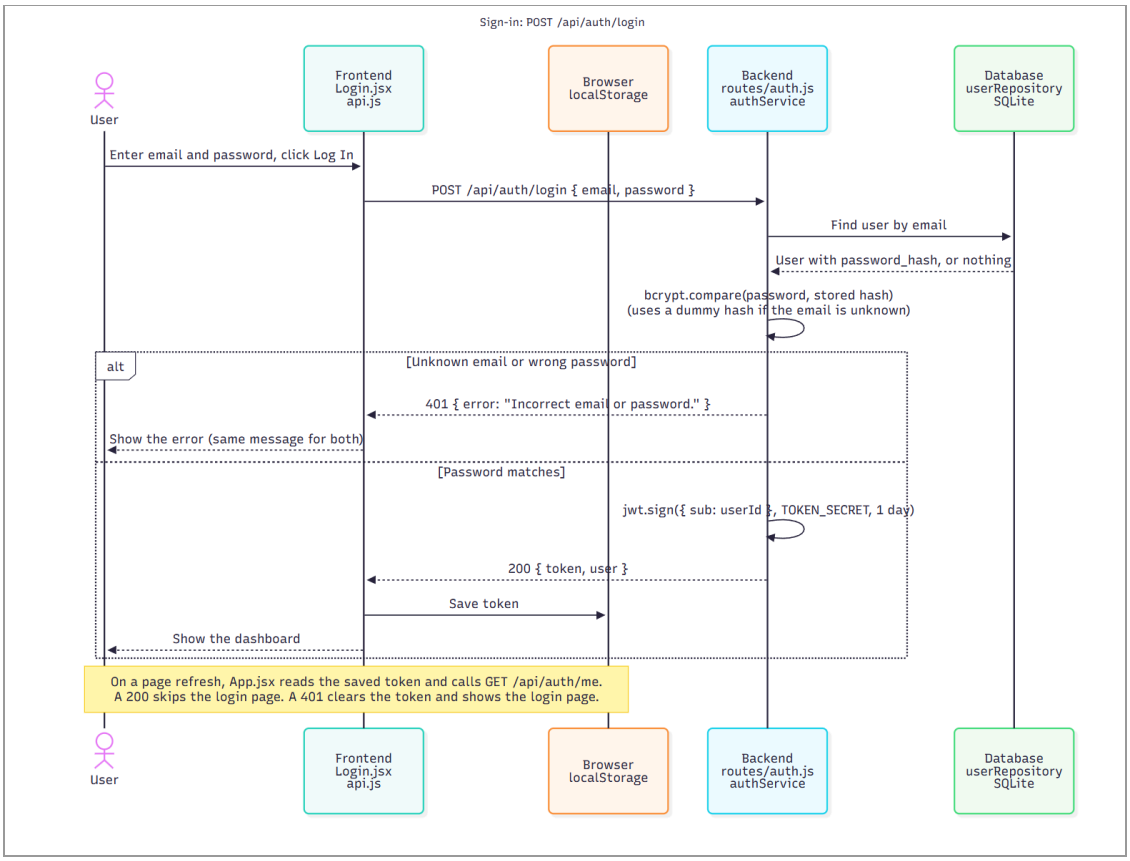
- **FAILURES:** A MISSING, INVALID, OR EXPIRED TOKEN RETURNS 401, AND THE FRONTEND THEN CLEARS THE TOKEN AND SHOWS THE LOGIN PAGE. ANOTHER USER'S RECORD RETURNS 404, SO THE API DOES NOT REVEAL THAT IT EXISTS. A WRONG PASSWORD AND AN UNKNOWN EMAIL RETURN THE SAME MESSAGE.
- **OVERVIEW (SIGN-IN AND ADDING A TRANSACTION):** DRAWN FROM THE REQUESTS THE RUNNING APP MAKES. IT REPLACES JUAN S.'S FIRST DRAFT (docs/architecture/sequence-overview-juan-s-draft.pdf), WHICH SHOWED A MICROSOFT SIGN-IN THE APP DOES NOT HAVE.


LedgerLogic: sign-in and adding a transaction (traced from the running app)
User
React frontend
Login.jsx
TransactionForm
api.js, token.js
Backend API
authenticate
routes
services
Database
repositories
SQLite
Sign-in
1. Enter email and password, select Log In
2. POST /api/auth/login { email, password }
3. Find user by email
4. User with password hash
5. bcrypt.compare,
sign a one-day JWT
alt
[Wrong email or password]
401 { error }
Show "Incorrect email or password."
[Correct]
6. 200 { token, user }
7. Save token
in localStorage
Add a transaction
8. Submit the income or expense form
9. POST /api/transactions
Authorization: Bearer token
10. Verify token,
set req.userId
alt
[Missing, invalid, or expired token]
401 { error }
Clear token,
show login page
[Valid token]
11. Find the category for this user
12. Category, or nothing
alt
[Invalid data or not this user's category]
400 { error, details }
Show the reasons on the form
[Valid data]
13. INSERT with user_id from the token
14. Saved record
15. 201 saved transaction
16. GET summary, transactions, categories,
and spending report (each with the token)
17. 200 with this user's data only
18. Totals, chart, bars, and list update
User
React frontend
Login.jsx
TransactionForm
api.js, token.js
Backend API
authenticate
routes
services
Database
repositories
SQLite

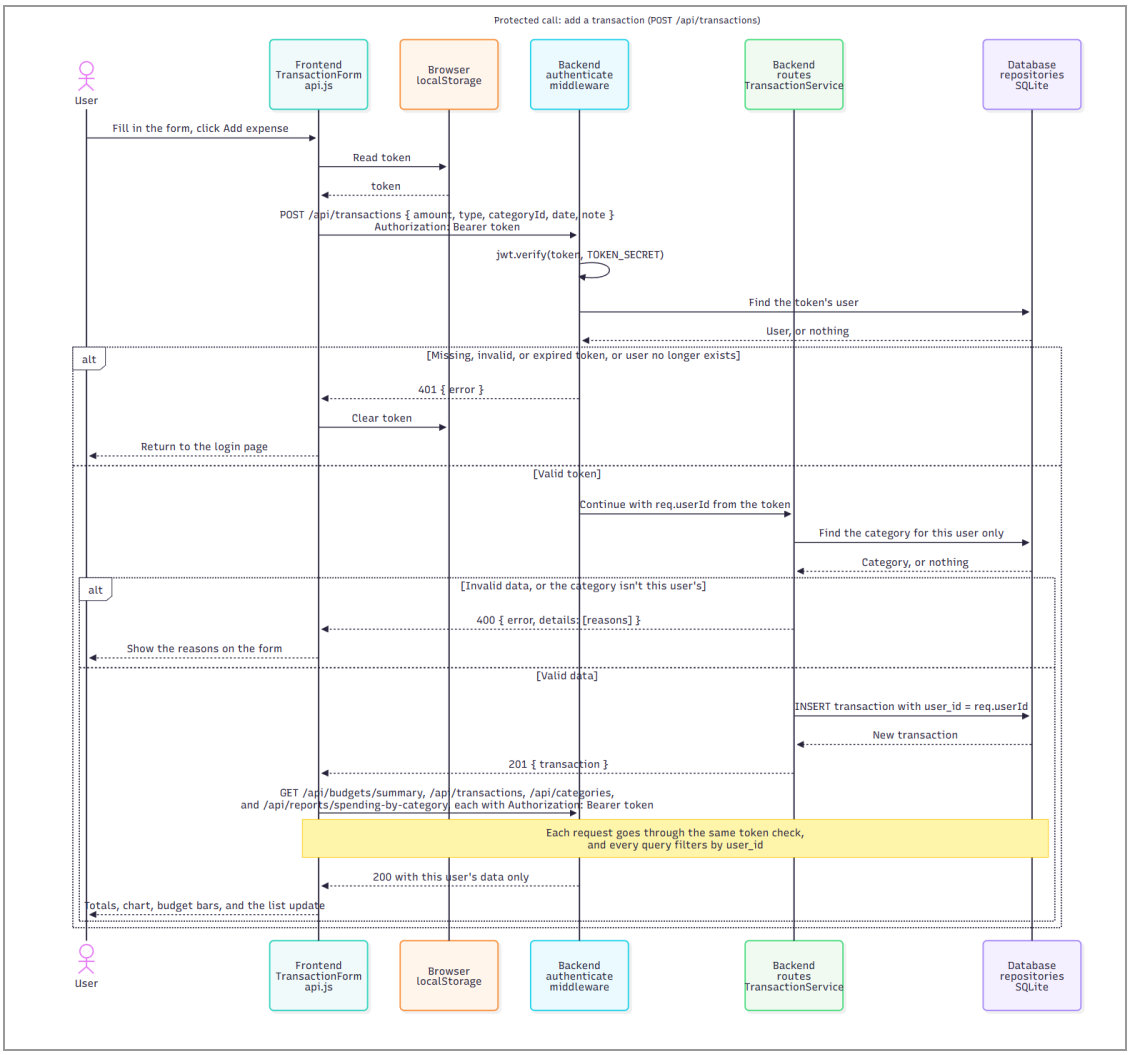
- **SIGN-UP FLOW:**



- **SIGN-IN FLOW:**



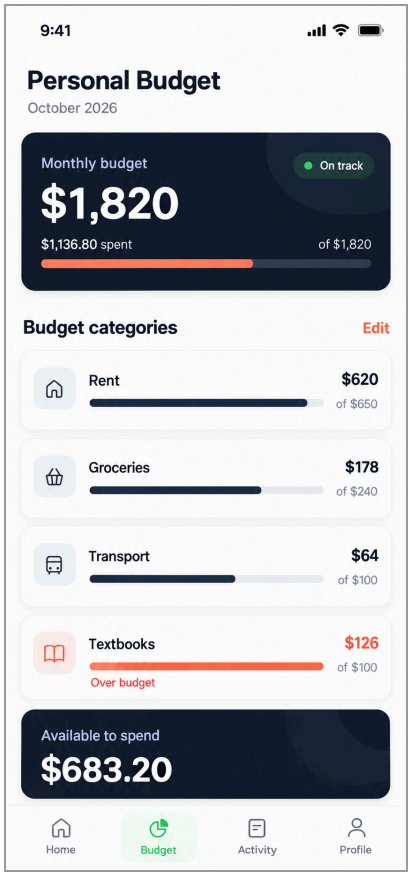
- **PROTECTED API CALL (ADD A TRANSACTION):**

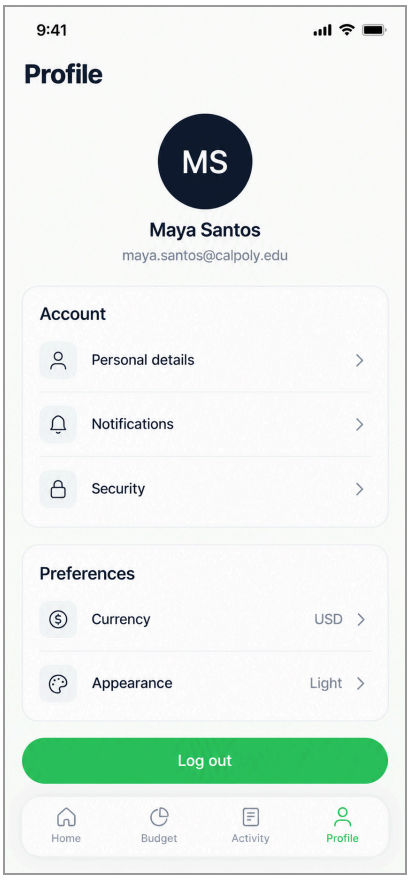


6. USER INTERFACE (UI)

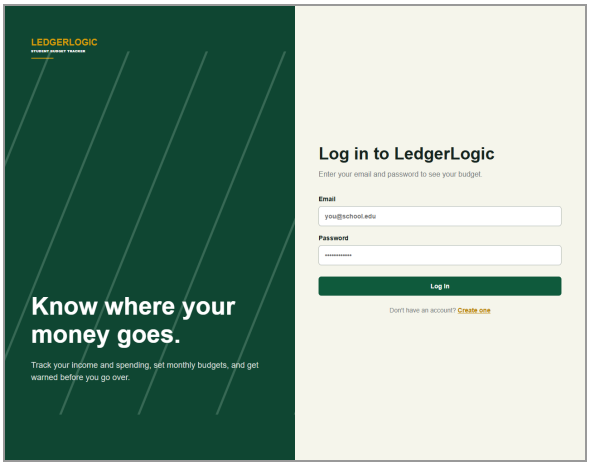
6.1 WIREFRAMES / MOCKUPS

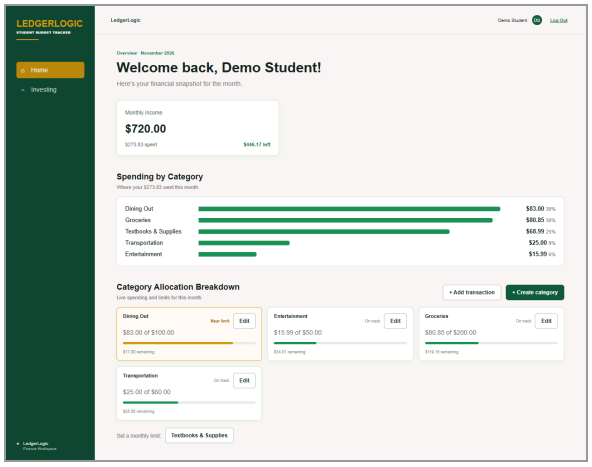
- **MOCKUPS (JUAN S.):** BUDGET SCREEN AND PROFILE SCREEN.





- **WORKING PROTOTYPE (JUAN F.):** LOGIN AND DASHBOARD SCREENS OF THE REACT APP. THE DASHBOARD SHOWS THE LOGGED-IN USER'S LIVE DATA FROM THE API, AND TRANSACTIONS AND CATEGORIES ARE ADDED AND EDITED THROUGH IT. THE INVESTING PAGES ARE A MOCKUP THAT DOES NOT SAVE DATA.





7. DATA REQUIREMENTS

7.1 LIST OF PERSISTENT DATA

- **USER ACCOUNT:** EMAIL (UNIQUE), PASSWORD HASH (NEVER PLAIN TEXT), DISPLAY NAME.
- **CATEGORY:** NAME (UNIQUE PER USER), TYPE (INCOME OR EXPENSE).
- **TRANSACTION:** AMOUNT (> 0), TYPE, CATEGORY, DATE (YYYY-MM-DD), NOTE (UP TO 200 CHARACTERS).
- **BUDGET:** CATEGORY, MONTH (YYYY-MM), LIMIT AMOUNT (> 0); ONE BUDGET PER CATEGORY PER MONTH.
- **NOT STORED (COMPUTED ON REQUEST):** MONTHLY TOTALS, REMAINING BALANCE, PERCENT OF BUDGET USED, BUDGET STATUS, SPENDING BY CATEGORY.

8. TRACEABILITY MATRIX

ID	REQUIREMENT	LINE OF CODE
US-01	Log a new expense with an amount, date, category, and description.	backend/src/routes/api.js:54; backend/src/services/transactionService.js:47; backend/src/domain/transaction.js:51 (validateTransaction); frontend/src/components/TransactionForm.jsx:12 (Add Transaction form)
US-02	Record paychecks and income sources.	backend/src/domain/transaction.js:20 (typeError: INCOME / EXPENSE); backend/src/routes/api.js:28; frontend/src/components/TransactionForm.jsx:70 (income / expense toggle)
US-03	Edit or delete a past transaction.	backend/src/routes/api.js:58 (PUT /transactions/:id), :62 (DELETE /transactions/:id); backend/src/services/transactionService.js:51 (update), :61 (remove); backend/src/repositories/transactionRepository.js:69, :82; frontend/src/components/TransactionList.jsx:12; frontend/src/components/TransactionRow.jsx:8 (Edit / Delete buttons);

		frontend/src/components/TransactionForm.jsx:12 (edit mode)
US-04	Dashboard with total income, total expenses, and remaining balance.	backend/src/routes/api.js:67; backend/src/services/budgetService.js:17; frontend/src/pages/Dashboard.jsx:13
US-05	Chart of spending by category.	backend/src/routes/api.js:71 (GET /reports/spending-by-category); backend/src/services/budgetService.js:48; backend/src/repositories/transactionRepository.js:39; frontend/src/components/SpendingChart.jsx:4; frontend/src/pages/Dashboard.jsx:93; frontend/src/api.js:78
US-06	Filter transaction history by category and date range.	backend/src/routes/api.js:45 (GET /transactions filters); backend/src/services/transactionService.js:20 (cleanFilters); backend/src/repositories/transactionRepository.js:49; frontend/src/pages/Transactions.jsx:75 (filter controls)
US-07	Create an account and log in with an email and password.	backend/src/routes/auth.js:7 (POST /auth/register), :11 (POST /auth/login); backend/src/services/authService.js:36 (register), :63 (login), :74 (userFromToken); backend/src/middleware/authenticate.js:4; backend/src/app.js:35 (protects every data route); backend/src/repositories/userRepository.js:1; backend/src/db/schema.sql:5 (users table); frontend/src/pages/Login.jsx:7; frontend/src/token.js:14 (setToken); frontend/src/api.js:25 (Bearer header), :34 (401 handling)

US-08	Set a monthly spending limit for individual categories.	backend/src/routes/api.js:75 (POST /budgets), :80 (PUT /budgets/:id), :32 (POST /categories); backend/src/services/budgetService.js:62; backend/src/services/categoryService.js:24; backend/src/domain/budget.js:26 (limitError); frontend/src/pages/CreateCategory.jsx:5; frontend/src/components/EditCategory.jsx:3; frontend/src/components/CategoryForm.jsx:13; frontend/src/pages/Dashboard.jsx:57 (categories with no limit yet)
US-09	Visual warning when spending exceeds 80% of a category's limit.	backend/src/domain/budget.js:2 (WARNING_THRESHOLD = 0.8); backend/src/domain/budget.js:16 (budgetStatus); frontend/src/pages/Dashboard.jsx:136 ("Near limit" / "Over budget" card)

9. APPENDICES

- **TE3 DOCUMENT (UML CLASS DIAGRAM, TRACEABILITY, MOCKUPS):** docs/te3/TE3-UML-Class-Diagram.pdf
- **TE4 DOCUMENT (ARCHITECTURE, HIGH-LEVEL DESIGN, DESIGN PATTERN, DECISIONS):** docs/te4/TE4-Design-and-Architectural-Patterns.pdf
- **TE5 REFACTORING TICKETS:** GITHUB ISSUES #50–#55 (LABEL "refactor"); ALL SIX ARE NOW IMPLEMENTED
- **TE6 ACCESS CONTROL:** SEQUENCE DIAGRAMS IN SECTION 5.5 AND ON THE README; SOURCES IN docs/architecture/
- **DEMO SCRIPT AND BACKUP SCREENSHOTS:** DEMO.md, docs/demo/
- **UNPLANNED WORK:** CHANGES OUTSIDE THE SPRINT PLAN THAT WERE NEEDED TO DEMO THE APP. SPRINT 4 (JUAN F.): BLANK-SCREEN FIX AND BROWSER DEMO DATA (#63, #64). SPRINT 5 (SHREEV B.): THE DEMO FIXES LISTED IN THE 11/06/2026 DOCUMENT HISTORY (#80).
- **DIAGRAM SOURCES:** docs/diagram-sources/, docs/architecture/
- **HOW TO RUN THE APP AND API REFERENCE:** README.md

10. AI USAGE & DISCLOSURE (MANDATORY)

- **MODEL(S) USED:** CLAUDE OPUS 5.5 (ANTHROPIC); GEMINI (GOOGLE)

- **PROMPTS USED DURING CODING/DOCUMENTATION:** "POPULATE OUR CSC 3100 SRS TEMPLATE WITH OUR TEAM'S PRODUCT VISION STATEMENT AND 9 USER STORIES."
- **TE3–TE4 (SHREEV B.):** CLAUDE (ANTHROPIC), USED THROUGH CLAUDE CODE, HELPED WRITE THE BACKEND AND ITS TESTS, GENERATE THE UML AND ARCHITECTURE DIAGRAMS, DRAFT THE TE3/TE4 DESIGN DOCUMENT, AND ORGANIZE THIS SRD UPDATE. THE TEAM REVIEWED, RAN, AND TESTED THE CODE AND CHECKED THE DOCUMENTS.
- **TE5–TE6 (SHREEV B.):** CLAUDE (ANTHROPIC), USED THROUGH CLAUDE CODE, HELPED WRITE THE REFACTORING, THE ACCESS CONTROL CODE AND ITS TESTS, THE SEQUENCE DIAGRAMS, AND UPDATES TO THIS DOCUMENT.